15-CONCURRENCY-AND-JOB-GRAPH.md — 4,000 files, zero races, always resumable

🔀 **Rule R2 — Every run is resumable.** Kill the process at any instant; restarting must continue the wedding, never restart it, and never corrupt the catalog. Everything in this file exists to make that sentence true.

1. The concurrency model (one model, no exceptions)

```
┌─ UI thread (Tauri/React) ── never blocks, never touches SQLite directly
│
├─ Orchestrator actor ────── owns the job graph, single-threaded state machine
│
├─ Writer actor ──────────── the ONLY writer to SQLite; bounded mpsc queue
│
├─ CPU pool (rayon) ──────── RAW decode, hashing, IO   | size = physical cores - 1
├─ GPU queue ─────────────── serialized submissions    | depth-limited, tile-budgeted
└─ Inference pool ────────── N sessions, N = f(VRAM)   | batched, never oversubscribed
```

Hard rules

- Exactly **one writer** to the catalog. Everything else sends messages.
- **No locks held across** `.await`, no `std::sync::Mutex` in async code, no blocking IO on the async runtime.
- **Bounded channels everywhere.** An unbounded channel is a memory leak with good manners. Backpressure is the mechanism that keeps RAM inside budget.
- **No shared mutable state between pools.** Data moves by ownership, not by reference-with-a-mutex.
- Lock ordering, where a lock is unavoidable, is globally fixed and documented: `project → photo → cache → gpu`. Acquiring out of order is a review rejection.

2. The job graph (frozen contract)

```
// crates/aura-jobs/src/contract/graph.rs — FROZEN CONTRACT v
1
#[derive(Debug, Clone, PartialEq, Eq, Hash, serde::Serialize,
serde::Deserialize)]
pub struct TaskId(pub String); // "{stage}:{photo_uuid}:{stag
e_version}" — deterministic

#[derive(Debug, Clone, Copy, PartialEq, Eq, serde::Serialize,
serde::Deserialize)]
pub enum TaskState { Pending, Ready, Running, Succeeded, Fail
ed, Quarantined, Skipped }

#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct Task {
    pub id: TaskId,
    pub stage: &'static str,
    pub photo_id: Option<i64>,
    pub deps: Vec<TaskId>,
    pub state: TaskState,
    pub attempt: u8,
    pub lease_expires_at: Option<i64>, // epoch ms; a crashed
```

```
worker's lease expires
    pub input_digest: String,          // blake3 of all input
s + stage version
    pub output_digest: Option<String>,
    pub cost_hint: CostHint,           // for scheduling, not
correctness
}

#[derive(Debug, Clone, Copy, serde::Serialize, serde::Deseria
lize)]
pub struct CostHint { pub cpu_ms: u32, pub gpu_ms: u32, pub r
am_mb: u32, pub vram_mb: u32 }
```

The graph lives in SQLite, not in memory. Memory holds a cache of it. **That is what makes resume free** — restart reads the graph and continues.

3. Idempotency — the rule that makes retries safe

Every task must satisfy: *running it twice produces the same state as running it once.*

- Task identity is content-derived (`input_digest`), so a re-run with unchanged inputs is a **cache hit**, not repeated work.
- All writes are `INSERT … ON CONFLICT DO UPDATE` keyed by `(task_id, input_digest)`.
- All file outputs are written to `path.tmp.<pid>` then `fsync` then atomically renamed.
- Side effects that cannot be idempotent (an email, an upload, a Postiz-style external post) are wrapped in an **outbox table** with a unique key and an at-least-once + dedupe design.

Idempotency test (mandatory per stage): run the stage, snapshot the catalog and the output tree, run it again, snapshot again — the snapshots must be identical.

4. Checkpointing and leases

- A task claims work by setting `state = Running` and `lease_expires_at = now + 90 s` in a single transaction.
- Long tasks heartbeat the lease every 30 s.
- On startup, any task with `state = Running` and an expired lease is reset to `Ready` with `attempt += 1`. Crash recovery is therefore just "open the catalog".
- After `attempt > max_attempts`, the task becomes `Quarantined` and its photo enters the review queue per `11-ERROR-TAXONOMY.md`.
- Checkpoints are written at stage boundaries, never mid-pixel; the WAL is checkpointed every 500 completed tasks or 30 s, whichever is first.

5. Cancellation and pause

- One `CancellationToken` per run, propagated to every task; checked at least every 250 ms of work and between every tile.
- Cancellation is **cooperative and clean**: the in-flight task finishes its current tile, releases its lease, and leaves the graph consistent. Nothing is left half-written.
- Pause = stop scheduling new tasks + let in-flight tasks drain. Resume = schedule again. The user can close the laptop mid-wedding.
- Hard requirement: cancel must take effect in **≤ 2 s** at p95, and the resulting catalog must pass `PRAGMA integrity_check`.

6. Scheduling policy

1. **Tier discipline** — never run a Tier-3 (full-res) task while any Tier-1 (analysis) task for the same wedding is pending. This is what delivers the sub-8-minute cull.
2. **Admission control** — a task is admitted only if `ram_mb` and `vram_mb` fit the live budget from `17-RESOURCE-BUDGETS.md`.
3. **Fair progress** — photos advance roughly in lockstep so progress is honest and cancellation loses little work.

4. **GPU serialisation** — one submission stream, queue depth ≤ 3, per-submission work sized to ≤ 2 s to avoid driver TDR.
5. **Deterministic tie-breaks** — when two tasks are equally ready, order by `TaskId`. Scheduling order must never affect results (see `12-DETERMINISM-CONTRACT.md`), but a stable order makes bug reports reproducible.

7. Progress reporting that does not lie

- Progress is computed from **completed cost**, not completed count — otherwise 90 % arrives in two minutes and takes forty more.
- The UI receives coalesced events at ≤ 10 Hz; it never subscribes per-photo.
- ETA uses a windowed throughput estimate and is displayed as a range, never a false-precision countdown.
- Progress is monotonic. If a re-edit loop adds work, the total rises visibly with an explanation rather than the bar sliding backwards.

8. Deadlock and race prevention

Hazard	Prevention	Detection
Lock cycles	Global lock order; prefer message passing	`parking_lot` deadlock detector in debug builds
Channel deadlock (full queue, blocked consumer)	All sends use `try_send` with a timeout ladder; consumers never send upstream	Watchdog: no task completion in 120 s → dump all task states
Data race	Ownership transfer; `Send` / `Sync` audited at crate boundaries	ThreadSanitizer job in CI
ABA / stale write	Optimistic concurrency: every row has `generation`; updates assert it	Conflict counter in telemetry
Torn multi-table write	One transaction per logical change	Kill -9 ladder in the chaos suite
Async task leak	Every spawn is tracked in a `TaskTracker`; shutdown joins all	Test asserts zero live tasks after run end

9. Test plan

- **Loom** — model-check the writer actor and the lease state machine under all interleavings.
- **ThreadSanitizer** — nightly job over the core crates.
- **Kill -9 ladder** — five kill points; final gallery must be identical to an uninterrupted run (this is also FMEA section 8).
- **Idempotency** — per-stage double-run snapshot equality.
- **Backpressure** — with a 4 GB RAM cap and a slow disk, RSS must stay under the ceiling and the run must still finish.
- **Cancellation storm** — start/cancel 200 times in a loop; no leaked threads, no leaked GPU resources, no lease left dangling.
- **Clock skew** — move the clock backwards 10 minutes mid-run; leases must not expire incorrectly (use a monotonic clock for leases).

10. Acceptance criteria

- [ ] Exactly one code path writes to SQLite; enforced by making the connection private to the writer actor.
- [ ] Zero unbounded channels in the repo — lint-enforced.
- [ ] Kill -9 ladder passes at all five points with identical galleries.
- [ ] Cancellation p95 ≤ 2 s; catalog integrity check clean afterwards.
- [ ] Loom and TSan jobs green.
- [ ] Every stage has a passing idempotency test.

11. Claude Code prompt

> Act as `SRC` (Senior Engineer — Core). Implement the job graph exactly as the frozen contract in `15-CONCURRENCY-AND-JOB-GRAPH.md`: SQLite-backed tasks, content-derived `TaskId`, leases with monotonic-clock expiry, the single writer actor, and bounded channels throughout. Then write the per-stage idempotency test harness. Then switch hats to `PERF` and prove backpressure holds RSS under the

> budget with a 4 GB cap. Report PASS/FAIL with command output. Do not introduce any unbounded channel or any second SQLite writer.